On-chain Task Tracker 前端架構技術規格書 (MVP 階段)

1. 專案願景與 MVP 開發範疇

1.1 架構設計導引

在 Web3 專案的初期開發中，建立一個穩健且具擴充性的前端架構具有決定性的戰略意義。前端不僅是使用者與區塊鏈互動的第一線，更是確保資料流動透明、操作流程直覺的核心。本規格書旨在定義 MVP（最小可行性產品）階段的前端藍圖，強調「功能聚焦」與「架構解耦」，以確保團隊能在不引入過度工程的前提下，實現快速迭代並為後續接入鏈上合約奠定基礎。

1.2 核心目標定義

前端作為 On-chain Task Tracker 的使用者入口點，在 MVP 階段肩負以下四大核心任務：

- 身份識別 **(Identity)**：提供流暢的錢包連接流程，作為鏈上身份的起始點。
- 資料呈現 **(Visibility)**：直觀展示任務列表與單一任務的即時狀態（MVP 階段先以卡片狀態呈現）。
- 行為發起 **(Action)**：提供建立任務的介面，將使用者意圖轉化為結構化資料。
- 狀態銜接 **(Bridging)**：預留擴充空間，以便未來無縫對接 API 或鏈上交易狀態 (On-chain Transaction Status)。

1.3 功能邊界評估 *(In-Scope vs. Out-of-Scope)*

為了確保開發時程受控並降低系統複雜性，本階段針對功能範圍進行了嚴格篩選：| 分類 | 本次 MVP 實作功能 (In-Scope) | 暫不實作功能 (Out-of-Scope) || ------ | ------ | ------ || 核心頁面 | 首頁 Dashboard、任務列表頁、建立任務頁 | AI 驗證畫面、深度鏈上資料分析 || 組件/導覽 | Header/Navbar、Wallet 狀態顯示區、任務概覽卡片 | 多錢包切換複雜流程、權限管理系統 || 激勵機制 | 基本狀態標籤（Created / Completed） | NFT / Token 獎勵發放介面 |
Architect's Analysis 範疇限制之戰略意義： 排除「AI 驗證」與「多錢包複雜流程」是基於開發戰略的考量。AI 驗證涉及複雜的第三方非同步邏輯，而多錢包切換則會大幅增加全域狀態管理的複雜度。透過聚焦於「主流程順暢度」，我們能將資源集中於核心 UI/UX 與資料模型的標準化，從而大幅縮短產品進入市場的時間 (Time-to-Market)。隨後，基於上述功能邊界，我們將定義支援這些操作的路由架構，確保 UI 狀態與導覽邏輯具備良好的依賴關係。

2. 頁面導覽與路由規劃 (Routing Architecture)

2.1 設計脈絡說明

清晰的路由導航是優化使用者路徑的關鍵。透過前端路由規劃，我們能確保 UI 狀態與 URL 同步，並讓使用者在「查看概況」與「執行操作」之間擁有流暢的切換體驗。

2.2 路由路徑定義

本專案初期規劃三條主要路徑，採用扁平化結構以簡化初期實作：

- **/ (**首頁**)**：入口點，提供全局視角。
- **/tasks (**任務列表頁**)**：任務管理的核心區域。
- **/tasks/new (**建立任務頁**)**：新任務產生的入口。**Architect's Analysis** 路由設計與業務價值： /tasks/new 具備最高的業務價值，因為它是鏈上資料產生的起點。目前採扁平路由以求快速交付，但在設計上已預留擴充性，未來可輕鬆改為 /tasks/:id 嵌套路由，以支援「查看單一任務細節頁」而無需調整導航邏輯。

2.3 頁面功能詳解

- 首頁 **(Dashboard)**：
- **Hero** 區塊：專案名稱與簡介，包含「查看任務」與「建立任務」的快速引導。
- **Wallet** 區塊：顯示錢包連線狀態，為 Web3 應用的核心識別。
- **Summary** 區塊：透過 TaskSummaryCard 顯示統計數據（Total / Completed / Pending）。
- **Recent Tasks** 區塊：展示最近三筆任務，提供快速存取。
- 任務列表頁：
- 顯示所有任務卡片，並預留篩選功能區塊。
- 單一任務狀態：在 MVP 階段，單一任務的狀態檢視將整合於 TaskCard 中，透過標籤（Created / Completed）讓使用者在列表頁即能掌握任務現況。在路由與頁面角色確立後，下一步需定義支撐這些頁面的元件層級，以實現組件化開發與跨頁面重用。

3. 元件階層與模組化策略 (Component Hierarchy)

3.1 元件化原則說明

我們遵循「關注點分離 (Separation of Concerns)」原則，將元件分為 Layout 類（結構定義）與功能類（邏輯實現）。這確保了 UI 的一致性，並降低了後續維護的難度。

3.2 Layout 類元件規格

- **AppLayout**：定義整體頁面框架，包含 Header 與 Main Content 容器。
- **Header**：包含 Logo、導覽選單及 Wallet 互動區塊。**Architect's Analysis** 狀態同步優勢：將 WalletConnect 整合於 Header 中，能確保使用者在切換不同路由時，錢包連線狀態始終可見且同步。這種集中式的連線管理，能有效避免頁面切換導致的狀態丟失或重複觸發 Provider 請求。

3.3 功能類元件規格

- **TaskSummaryCard**：純展示元件，接收並顯示統計數據。
- **TaskCard**：核心顯示單元。重用性設計：此元件設計為 Single Source of Truth，同時應用於首頁的「最近任務」與任務列表頁，確保視覺邏輯一致。
- **TaskForm**：包含標題與描述輸入欄位，負責蒐集使用者資料並觸發提交邏輯。**Architect's Analysis** 生命週期引導：TaskCard 透過狀態標籤的設計，不僅是視覺美化，更是為了引導使用者理解任務的生命週期。從「剛建立 (Created)」到「已完成 (Completed)」的視覺回饋是提升 Web3 應用使用者信心的關鍵。

4. 專案目錄結構 (Folder Structure)

為了維持架構的整潔與可維護性，本專案將採用以下結構進行 Scaffolding：

```
src/
├── assets/              # 靜態資源 (Images, Global CSS)
├── components/          # 可重用 UI 元件
│   ├── layout/          # AppLayout, Header
│   ├── shared/          # Button, Card, Badge
│   └── tasks/           # TaskCard, TaskForm, TaskSummaryCard
├── context/             # 全域狀態管理 (WalletContext)
├── hooks/               # 自定義 Hook (例如 useWallet, useTasks)
├── mocks/               # 開發初期 Mock 資料 (tasks.ts)
├── pages/               # 頁面級元件 (Home, TaskList, CreateTask)
```

```
├── routes/              # 路由配置定義
├── types/               # TypeScript 介面定義 (task.d.ts,
wallet.d.ts)
└── utils/               # 通用工具函式 (格式化地址, 時間等)
```

5. TypeScript 資料模型定義 (Data Modeling)

5.1 型別安全戰略說明

在處理鏈上資料時，預先定義強型別模型能有效降低執行時期錯誤。這不僅提升開發效率，更為未來銜接 Smart Contract 的返回值提供了清晰的對接標準。

5.2 核心 *Interface* 定義

```
// 任務核心型別
export interface Task {
  id: string;
  title: string;
  description: string;
  status: 'Created' | 'Completed';
  createdAt: string; // 採用 ISO8601 string 格式，確保跨環境時間解析一致
  transactionHash?: string; // 預留欄位，用於未來儲存鏈上交易雜湊
}

// 錢包狀態型別
export interface WalletState {
  isConnected: boolean;
  address: string | null;
}

// 統計數據型別
export interface TaskSummary {
  total: number;
  completed: number;
  pending: number;
}
```

Architect's Analysis 鏈上對接擴充性：在 Task 型別中加入 transactionHash 是為了銜接「鏈上交易狀態」。一旦未來切換至合約互動，前端即可透過此 Hash 追蹤交易是否被 Confirmed，進而動態更新 UI 狀態。

5.3 資料模擬 *(Mocking)* 策略

初期開發將依賴 src/mocks/tasks.ts。這讓 UI 開發能與合約開發解耦，待合約部署後，只需調整資料獲取層即可完成串接。

6. 狀態管理與資料流 (State Management & Data Flow)

6.1 技術決策說明

在 MVP 階段，我們選擇 **React Context API** 作為主要狀態管理工具。相較於 Redux，Context API 顯著減少了樣板代碼 (Boilerplate)，並能有效控制 Bundle Size，對於目前的錢包狀態管理已綽綽有餘。

6.2 Context 實作細節

- **WalletContext**：管理全域錢包連線狀態，並提供地址縮寫工具。

6.3 資料存取演進路徑 *(Evolution Blueprint)*

Architect's Analysis 從 **Mock** 到 **Web3** 的切換成本：當前採用 useEffect 讀取 Mock 資料。演進藍圖如下：

1. **Phase 1 (Mock)**：直接導入靜態文件。
2. **Phase 2 (Contract Interaction)**：引入 wagmi 或 ethers.js 庫。
3. **Phase 3 (State Sync)**：將資料獲取邏輯封裝入 Custom Hooks，元件層級不需改動任何 UI 邏輯即可完成遷移。

7. UI/UX 風格規範與實作時程 (Implementation Roadmap)

7.1 視覺風格導引

對於 Web3 專業工具而言，**「乾淨、卡片式、響應式」**是設計的核心。我們將採用白底黑字的高對比風格，確保資訊層級清晰，降低使用者的認知負擔。

7.2 開發階段規劃

1. 專案啟動：建立 React + TypeScript + Vite 專案。
2. 結構初始化：依據本規格書建立目錄結構。
3. 基礎建設：實作前端路由、AppLayout 與 Header。
4. 核心邏輯：建立 WalletContext 與基礎型別。
5. 頁面開發：開發 HomePage 並確保元件可重用。
6. 資料對接：完成首頁並實現動態 Mock 資料顯示。**Architect's Analysis Vite** 之戰略價值：採用 Vite 提供快速的熱更新 (HMR)，能極大提升 MVP 開發階段的迭代速度。**Definition of Done (DoD)**：階段 6 完成後，所有 UI 元件必須能成功透過定義的 TypeScript 介面注入 Mock 資料，確認資料流邏輯與未來合約 API 格式完全相容。

8. 結語與交付清單

本文件確立了 On-chain Task Tracker 前端的架構基礎。完成此階段後，我們將獲得以下成果：

1. 明確的頁面導覽定義
2. 標準化的元件拆解與重用規範
3. 強型別的資料模型 **(TypeScript)**
4. 清晰的專案目錄結構規劃架構設計已完備，團隊可立即進入實作開發階段。